

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Grafos Dirigidos

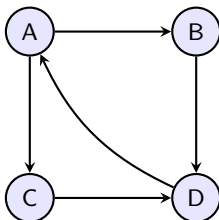
Grafos dirigidos

En la clase anterior trabajamos con **grafos no dirigidos**, donde las relaciones eran **simétricas**: si A está conectado con B, B lo está con A.

Sin embargo, en muchos contextos reales las conexiones son **asimétricas** y tienen un sentido definido:

- Un banco *otorga* un préstamo a un cliente.
- Un usuario *sigue* a otro en una red social.
- Un sitio web *enlaza* a otro.

Para representar estas relaciones con dirección, utilizamos **grafos dirigidos**.



Ejemplo

Ejemplo: flujo de envíos entre distintas plantas, centros de distribución y sucursales de una empresa.

Cada **nodo** es una **planta o centro logístico**, y una **arista** dirigida (u, v) indica que la planta u textbfenvía productos al destino v .

Los datos del flujo de envíos pueden almacenarse en un archivo CSV `produccion.csv` con el siguiente formato:

```
PlantaNorte,CentroDistribucion
PlantaNorte,SucursalCABA
PlantaSur,CentroDistribucion
CentroDistribucion,SucursalCABA
CentroDistribucion,SucursalCordoba
CentroDistribucion,SucursalRosario
```

El grafo asociado es dirigido, ya que los envíos tienen un sentido definido.

Crear un grafo dirigido desde un CSV

La clase `DiGraph` de la biblioteca `networkx` se usa para representar grafos dirigidos.

Un objeto `DiGraph` puede crearse a partir de un CSV.

```
import networkx as nx

def cargar_flujo(nombre_archivo):
    """Carga los datos desde un archivo CSV y construye el grafo dirigido."""
    G = nx.read_edgelist(
        nombre_archivo,
        delimiter=";",
        create_using=nx.DiGraph() # grafo dirigido
    )
    return G

G = cargar_flujo("produccion.csv")
```

El objeto `G` es un grafo dirigido (`DiGraph`) con las plantas y los flujos logísticos.

Predecesores y sucesores de un nodo

En un grafo dirigido:

- Los **sucesores** de un nodo son aquellos a los que apunta.
- Los **predecesores** son los que lo apuntan.

Los métodos `G.successors(n)` y `G.predecessors(n)` permiten acceder a los nodos sucesores y predecesores de `n`, respectivamente.

```
nodo = "CentroDistribucion"

print("Sucesores de", nodo, ":", list(G.successors(nodo)))
print("Predecesores de", nodo, ":", list(G.predecessors(nodo)))
```

Salida esperada:

```
Sucesores de CentroDistribucion : ['SucursalCABA', 'SucursalCordoba',
'SucursalRosario']
Predecesores de CentroDistribucion : ['PlantaNorte', 'PlantaSur']
```

Grado de entrada y salida

En un grafo dirigido:

- El **grado de salida** (*out degree*) de un nodo indica cuántas aristas **salen** de él.
- El **grado de entrada** (*in degree*) indica cuántas aristas **llegan** a él.

Estos valores pueden obtenerse con los métodos `G.out_degree(n)` y `G.in_degree(n)`.

```
nodo = "CentroDistribucion"  
print("Grado de salida de", nodo, ":", G.out_degree(nodo))  
print("Grado de entrada de", nodo, ":", G.in_degree(nodo))
```

Salida esperada:

```
Grado de salida de CentroDistribucion : 3  
Grado de entrada de CentroDistribucion : 2
```

Visualizar grafos dirigidos

Podemos dibujar el grafo usando flechas para representar el sentido de los envíos.

```
import matplotlib.pyplot as plt

def dibujar_flujo(G):
    """Dibuja el grafo con etiquetas y flechas."""
    pos = nx.spring_layout(G, seed=42)
    nx.draw(
        G, pos,
        with_labels=True,
        node_color="lightblue",
        node_size=1800,
        arrowsize=15
    )
    plt.show()

dibujar_flujo(G)
```

Ejercicio: red de transferencias bancarias

Ejercicio 1. Se dispone de información sobre transferencias electrónicas entre distintas cuentas bancarias. Cada nodo representa una **cuenta**, y una arista dirigida (u, v) indica que la cuenta u **transfirió dinero** a la cuenta v .

El archivo `transferencias.csv` (sin encabezado) tiene el siguiente formato:

```
CuentaA , CuentaB
CuentaA , CuentaC
CuentaB , CuentaC
CuentaC , CuentaD
CuentaD , CuentaA
CuentaE , CuentaC
...
```

Se pide:

- (a) Crear un grafo dirigido G a partir del archivo `transferencias.csv`.
- (b) Dibujar el grafo con flechas que indiquen el sentido de las transferencias.
- (c) Para cada cuenta, mostrar quiénes le transfieren y a quiénes transfiere.
- (d) Calcular la cantidad de transferencias hechas desde cada cuenta.
- (e) Calcular la cantidad de transferencias recibidas por cada cuenta.

Grafos dirigidos ponderados

Hasta ahora trabajamos con grafos dirigidos donde las aristas sólo indicaban la **existencia** de una relación. En muchos casos, las relaciones tienen además un **valor** asociado.

Por ejemplo, en una red logística puede registrarse cuántos envíos se realizaron entre cada par de centros:

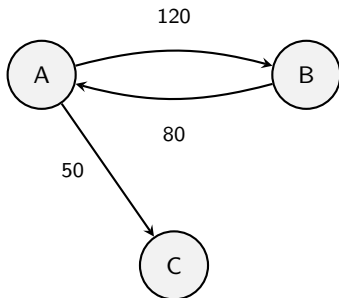
- (A, B) : 120 envíos desde A hacia B.
- (B, A) : 80 envíos desde B hacia A.
- (A, C) : 50 envíos desde A hacia C.

A este tipo de grafos los llamamos **grafos ponderados**, y la información cuantitativa se almacena como un **atributo de las aristas**.

Grafos dirigidos ponderados

Hasta ahora trabajamos con grafos dirigidos donde las aristas sólo indicaban la **existencia** de una relación. En muchos casos, las relaciones tienen además un **valor** asociado.

Por ejemplo, en una red logística puede registrarse cuántos envíos se realizaron entre cada par de centros:



Ejemplo: red de envíos ponderada

El archivo `envios.csv` (sin encabezado) contiene información sobre la cantidad de envíos entre centros:

```
CentroA,CentroB,120
CentroB,CentroA,80
CentroA,CentroC,50
CentroC,CentroA,30
CentroB,CentroC,20
CentroC,CentroD,40
CentroD,CentroB,10
```

Cada fila representa una arista dirigida (u, v) con un **peso** asociado.

El objetivo es construir un grafo dirigido ponderado con esta información.

Crear un grafo dirigido con pesos desde CSV

networkx permite leer aristas con pesos utilizando el parámetro data.

```
import networkx as nx

def cargar_envios(nombre_archivo):
    """Carga los datos desde un archivo CSV y construye un grafo dirigido con pesos."""
    G = nx.read_edgelist(
        nombre_archivo,
        delimiter=",",
        create_using=nx.DiGraph(),
        data=[("cant_envios", int)] # atributo de peso
    )
    return G

G = cargar_envios("envios.csv")
```

El objeto G es un grafo dirigido ponderado (DiGraph) donde cada arista tiene un atributo cant_envios que indica la cantidad de envíos.

Trabajar con pesos en grafos dirigidos

En un grafo dirigido ponderado, cada arista tiene uno o más **atributos** asociados (por ejemplo: cantidad de envíos, tiempo de viaje, costo logístico).

En `networkx`, estos atributos se almacenan como un diccionario:

```
G["CentroA"]["CentroB"] # -> {"cant_envios": 120}
```

Podemos acceder a todos los atributos de una arista:

```
for u in G:
    for v in G[u]:
        print(u, "->", v, ":", G[u][v]["cant_envios"])
```

Esto permite trabajar con información cuantitativa asociada a las relaciones dirigidas.

Dibujar grafos dirigidos ponderados

Para visualizar un grafo ponderado, podemos agregar etiquetas en las aristas mostrando el peso.

```
import matplotlib.pyplot as plt

def dibujar_envios(G):
    """Dibuja un grafo dirigido mostrando cantidades de envíos."""
    pos = nx.spring_layout(G, seed=42)

    nx.draw(
        G, pos,
        with_labels=True,
        node_color="lightyellow",
        node_size=1600,
        arrowsize=15
    )

    # Obtener diccionario de etiquetas: {(u,v): peso}
    etiquetas = nx.get_edge_attributes(G, "cant_envios")
    nx.draw_networkx_edge_labels(G, pos, edge_labels=etiquetas)
    plt.show()

dibujar_envios(G)
```

Este método permite visualizar tanto la dirección como la magnitud del flujo entre nodos.

Ejercicio: red de movilidad urbana

Ejercicio 2. Una ciudad registra la cantidad de pasajeros que se trasladan diariamente entre distintas estaciones de su red de transporte. Cada nodo representa una **estación**, y una arista dirigida (u, v) indica que desde u se trasladan pasajeros hacia v . El peso en cada arista indica la **cantidad promedio de pasajeros por día**.

El archivo `pasajeros.csv` (sin encabezado) tiene este formato:

```
A,B,4500
B,A,3800
A,C,2100
C,A,1800
B,C,900
C,D,1600
...
```

Se pide:

- Crear un grafo dirigido ponderado G a partir del archivo.
- Dibujar el grafo mostrando la cantidad de pasajeros sobre cada arista.
- Para cada estación, mostrar:
 - cuántos pasajeros salen de la estación;
 - cuántos pasajeros llegan a la estación.
- Determinar cuál es la estación con mayor tráfico diario de pasajeros, considerando la suma de pasajeros que llegan y de pasajeros que parten.

Repaso de la clase

En esta clase trabajamos con **grafos dirigidos**, donde cada arista tiene un sentido.

Resumen:

- Grafos dirigidos.
- Grafos dirigidos ponderados.

Con lo visto, ya pueden completar hasta el ejercicio 4.4 de la guía de Métodos Analíticos y Programación.